

Simulazione di uno stormo inseguito da un predatore

Mila Casali, Francesco Fonseca

Introduzione

Gli stormi di volatili sono in grado di spostarsi in modo coordinato nella stessa direzione, senza la presenza di un capo o di una figura di autorità che li guidi o organizzi. Dunque il nostro scopo è riuscire a simulare uno stormo che si comporti in modo ordinato e coeso partendo dall'implementazione del moto di un singolo volatile. Il problema di riuscire ad emulare tale comportamento venne affrontato da Craig Reynolds nel 1986 [1], che propose il modello al quale ci rifacciamo in questo progetto. Tale modello prende il nome dal software ideato da Reynolds che venne battezzato "boids".

1. Analisi del problema

La simulazione avviene in uno spazio tridimensionale toroidale e si basa sul fatto che ogni singolo ente segua tre regole fondamentali ideate dallo stesso Reynolds [2]: la separazione, che fa in modo che ogni uccello eviti di avvicinarsi eccessivamente ai vicini, l'allineamento, che spinge ogni volatile ad adattare la propria velocità a quella dei compagni e infine la coesione, che mantiene l'unità dello stormo facendo in modo che ogni uccello vada verso il centro dello stesso. In aggiunta a queste tre regole basilari abbiamo implementato la simulazione in modo tale che tenga conto della presenza di un predatore dal quale gli uccelli fuggono e in modo tale che il moto dello stormo sia influenzato dalla presenza del vento che spira in una specifica direzione, inoltre lo stormo tende a mantenersi a una determinata quota. Ognuna di queste indicazioni è coadiuvata da fattori limitanti sulla velocità e posizione dei singoli volatili in modo tale da avere un comportamento realistico, tali fattori potranno essere inseriti dall'utente o generati casualmente, essi includono: l'ampiezza dello spazio di simulazione, l'apertura alare, la velocità massima, la distanza minima, i fattori di allineamento, coesione, separazione e paura del predatore, la velocità del vento e la sua direzione, la velocità e il raggio di attacco del predatore. Inoltre saranno sempre a discrezione dell'utente la presenza del vento e dello spazio toroidale.

2. Progettazione dell'Algoritmo

Il problema è stato affrontato implementando un algoritmo che tenga conto dei dati relativi a uno stormo di boids e allo stesso tempo li utilizzi per determinare il comportamento di ogni singolo boid in ottemperanza alle regole sopracitate.

2.1 Scelte di Progetto

Come ausilio allo sviluppo del programma abbiamo utilizzato la repository gitHub riportata in appendice [3], nella quale sono state caricate gradualmente la maggior parte delle versioni del codice. Per quanto riguarda l'ambiente in cui si muovono i boids abbiamo scelto di implementare 2 possibili strategie: un ambiente con confini rigidi che determinano un "rimbalzo" dei boid e uno spazio toroidale per far sì che i volatili si muovano senza confini rigidi. La scelta del tipo di ambiente è effettuata dall'utente.

Relativamente alla simulazione grafica del volo, abbiamo utilizzato la libreria grafica SFML per gestire la visualizzazione della simulazione proiettata in 2D su 2 piani (XY e XZ).

2.2 Passi Principali dell'Algoritmo

L'algoritmo che regola il movimento dello stormo consiste nei seguenti passi:

1. Inizializzazione dello stormo: generazione delle posizioni e delle velocità iniziali dei boid.
2. Calcolo delle forze: per ogni boid, in base alle posizioni e alle velocità dei boids vicini, calcolo delle forze di separazione, allineamento e coesione.
3. Influenza ambientale: applicazione delle forze derivanti dal vento e dalla presenza del predatore.
4. Aggiornamento del movimento: aggiornamento delle velocità e delle posizioni dei boid.
5. Gestione dei bordi: applicazione delle regole toroidali o di rimbalzo ai confini dello spazio di simulazione.
6. Interazione con il predatore: il predatore cerca di catturare i boid, che a loro volta tentano di fuggire.
7. Estrazione delle statistiche: ad ogni unità di tempo (o multipli) vengono estratte una serie di misure che riassumono il comportamento dello stormo.

3. Implementazione

Il progetto è organizzato modularmente, suddiviso in un file sorgente e una serie di file di intestazione che contengono le definizioni delle classi e le funzioni ausiliarie. Per quanto riguarda le classi ausiliarie si è scelto di includere sia la dichiarazione sia l'implementazione nel file header per semplificare la gestione del codice. Il file principale, '**boids.cpp**', funge da entry point per l'intero programma e coordina l'interazione tra le varie classi e funzioni, gestisce l'inizializzazione dei parametri della simulazione, il salvataggio delle statistiche e l'aggiornamento in tempo reale delle finestre grafiche

Il file principale deve essere compilato con il comando:

```
g++ boid.cpp functions.cpp predator.cpp statistics.cpp swarm.cpp
variables.cpp vec3.cpp boids.cpp -Wall -Wextra -Wpedantic -Wconversion
-Wsign-conversion -Wshadow -Wimplicit-fallthrough -Wextra-semi
-Wold-style-cast -D_GLIBCXX_ASSERTIONS -fsanitize=address -o boids
-lsfml-graphics -lsfml-window -lsfml-system
```

allo scopo di richiamare correttamente tutte le librerie grafiche.

Oltre al file principale, ci sono alcuni file di libreria che contengono le classi fondamentali per il funzionamento del programma. Questi file sono:

- **vec3.cpp**: Contiene la definizione della classe 'Vec3', che rappresenta i vettori tridimensionali utilizzati per gestire le posizioni e le velocità dei boids nello spazio. Questa classe include operazioni vettoriali essenziali per il movimento e l'interazione dei boids.
- **boid.cpp**: Contiene la definizione della classe 'Boid', l'unità di base della simulazione, che rappresenta i singoli volatili dello stormo. Ogni boid ha come proprietà una posizione e una velocità. Sono inoltre presenti metodi volti all'aggiornamento delle velocità e delle posizioni dei boids.
- **predator.cpp**: Definisce la classe 'Predator', una specializzazione di 'boid' che introduce un predatore nello stormo. Questo file gestisce le caratteristiche uniche del predatore, come la capacità di cacciare i boids, aggiungendo un ulteriore livello di complessità alla simulazione.

- **swarm.cpp**: Definisce la classe ‘Swarm’, che gestisce lo stormo di boids. Questa classe è responsabile di aggiornare lo stato di ogni boid e di regolare le interazioni tra di essi, oltre a gestire la logica globale dello stormo.

I File di Utility e le Variabili di Configurazione sono inclusi nelle seguenti librerie:

- **functions.cpp**: Include funzioni ausiliarie e utility che supportano la logica principale del programma. Queste funzioni possono gestire l’interfaccia utente, inizializzazione dei parametri, calcoli matematici o altre operazioni necessarie per il corretto funzionamento della simulazione.
- **statistics.cpp**: Contiene funzioni necessarie per ottenere i dati relativi alla simulazione, come le distanze, le medie di velocità e posizione e le loro relative deviazioni standard, oltre a una funzione che gestisce il comportamento ai bordi.
- **variables.cpp**: Definisce e inizializza le strutture dati che contengono i parametri di configurazione globali per la simulazione.

Il testing della libreria è realizzato tramite i seguenti file:

- **boids.test.cpp**: Questo file contiene l’implementazione dei test unitari per verificare il corretto funzionamento delle principali classi e funzioni del progetto. Utilizza la libreria Doctest, un framework di testing leggero e ricco di funzionalità per C++.

3.1 Descrizione dettagliata delle classi fondamentali

3.1.1 La classe “Vec3”

La classe ‘Vec3’ rappresenta un vettore tridimensionale e costituisce una componente fondamentale del progetto, fornendo una struttura dati essenziale per la gestione delle operazioni geometriche nello spazio 3D. Questa classe implementa vari metodi per operazioni vettoriali di base, come la somma, la sottrazione, il prodotto scalare e vettoriale, la normalizzazione e il calcolo della norma del vettore. Inoltre, la classe include alcune operazioni necessarie al calcolo delle distanze in uno spazio toroidale. Ogni operazione è implementata tramite operatori. Ad esempio, la somma di due vettori ‘Vec3’ può essere eseguita utilizzando l’operatore ‘+’, mentre la moltiplicazione per uno scalare è realizzata tramite l’operatore ‘*’. La classe include anche un metodo per accedere agli elementi del vettore tramite l’overload dell’operatore ‘[]’.

3.1.2 La classe “Boid”

La classe ‘Boid’ rappresenta l’unità base del sistema di simulazione, modellando un singolo volatile che si muove nello spazio tridimensionale. Ogni boid è caratterizzato da una posizione (‘position’) e una velocità (‘velocity’), entrambe rappresentate da un vettore di classe ‘Vec3’. La classe fornisce metodi per ottenere e modificare questi attributi, consentendo di aggiornare la posizione e la velocità del boid in base alle interazioni con altri boids e con l’ambiente circostante. Il metodo ‘updateBoidVelocity’ aggiorna la velocità del boid tramite un’accelerazione e successivamente il metodo ‘updateBoid’ ne aggiorna la posizione considerando la nuova velocità. Se la velocità supera un limite massimo, viene normalizzata e ridimensionata per rientrare in questo limite.

3.1.3 La classe “Predator”

La classe Predator è una sottoclasse di ‘Boid’ che, in aggiunta, contiene due nuovi attributi, ossia la velocità massima ‘attack_speed’ e il raggio di attacco ‘attack_range’. La classe contiene inoltre metodi specifici per la ricerca e l’attacco di una preda all’interno di uno swarm, rispettivamente ‘findPrey’, che restituisce un puntatore ad un boid dello ‘Swarm’, e ‘attack’, che modifica la velocità del predatore per indirizzarlo alla preda più vicina. Il predatore insegue la preda più vicina e, nel momento in cui riesce a catturarla, il cooldown della classe Swarm si azzerà e il predatore torna alla sua quota preferenziale, ossia $\frac{1}{3}$ dello schermo, grazie alla funzione ‘maintainHeight’, definita in ‘statistics.hpp’ e discussa più nel dettaglio in seguito, viaggiando a una velocità in una direzione casuale. Dopo un certo lasso di tempo, il predatore torna all’attacco lanciandosi verso lo stormo e cercando una nuova preda. Alla fine, i boid catturati vengono rimossi dallo stormo.

3.1.4 La classe “Swarm”

La classe ‘Swarm’ è il fulcro del progetto, essa rappresenta uno stormo di boid e gestisce la loro dinamica di movimento, regolata da vari parametri come la velocità massima, la distanza minima tra i boid, e fattori di separazione, coesione, allineamento e paura.

La posizione iniziale dei boid viene generata casualmente all’interno di un’area di schermo definita dal vettore ‘screen’. Per ogni boid, vengono generate tre coordinate casuali (x, y, z) che rappresentano la posizione iniziale del boid nello spazio tridimensionale definito dal vettore screen. La velocità iniziale del boid viene generata creando tre componenti (vx, vy, vz) calcolate come numeri casuali compresi tra $-\text{max_speed}/\sqrt{3}$ e $\text{max_speed}/\sqrt{3}$. Ciò garantisce che la velocità iniziale sia casuale ma limitata da ‘max_speed’.

Oltre alle regole di movimento, la classe gestisce anche altri fattori, come la presenza di vento (‘wind’), che influenza il movimento dei boid. Il metodo principale ‘updateSwarm’ aggiorna la posizione e velocità di tutti i boid tenendo conto delle regole di movimento, del predatore, e delle condizioni ambientali. Se un boid si trova all’interno del raggio di azione del predatore (‘wingspan’), viene rimosso dallo stormo, simulando la sua cattura. Per i boid non catturati, vengono calcolate nuove velocità basate sulle regole di comportamento: viene richiamato due volte il metodo di ‘Boid’, ‘updateBoidVelocity’ passando come ‘delta_v’ prima la somma delle correzioni alla velocità dovute alle 4 regole, poi quella relativa alla fuga dal predatore, per aumentarne il peso. Viene poi richiamato il metodo ‘updateBoid’ passando come ‘delta_v’ la correzione dovuta al vento, questo metodo oltre alla velocità modifica la posizione del boid tenendo in considerazione la velocità precedentemente modificata e la correzione dovuta al vento.

Infine viene richiamata una funzione ‘border’ allo scopo di mantenere il boid nei confini designati: essa rimanda a due funzioni ‘teleport’ e ‘bounce’ che gestiscono rispettivamente lo spazio toroidale e i bordi.

Il volo dei boid segue tre regole principali: tendenza verso il centro di massa dello stormo (funzione ‘cohesion’), mantenimento di una distanza minima dagli altri boid (‘separation’ e ‘bounce’), e allineamento con la velocità media degli altri boid vicini (‘alignment’), con possibilità di applicare un comportamento toroidale (teletrasporto attraverso i bordi dello schermo) o limitato dai bordi. Altri comportamenti fondamentali sono la paura del predatore (‘fear’) e la tendenza a mantenere un’altitudine preferenziale (‘maintainHeight’).

coesione. La funzione ‘cohesion’ implementa la regola che tende a tirare ogni boid verso il centro di massa percepito dello stormo. L’obiettivo è avvicinare ogni boid agli altri per mantenere la coesione del gruppo.

Nella funzione, il centro di massa percepito ('perceived_center') viene inizialmente impostato su un vettore nullo. La funzione scorre poi tutti i boid nello stormo, calcolando la posizione media di quelli che si trovano all'interno di una certa distanza ('sight_distance') rispetto al boid attualmente considerato ('b'). Se il boid in esame non è troppo lontano (cioè se è entro la 'sight_distance'), la sua posizione viene aggiunta al centro di massa percepito, e il conteggio ('count') dei boid vicini viene incrementato. Se non ci sono boid vicini (ossia, 'count' è zero), la funzione restituisce un vettore nullo ('vec3(0, 0, 0)'), indicando che non c'è alcun movimento verso il centro di massa. Se ci sono boid vicini, il centro di massa percepito viene calcolato facendo la media delle posizioni di questi boid vicini, e il boid corrente ('b') viene spinto leggermente verso questo centro di massa percepito. La forza di questa spinta è scalata da un fattore di coesione ('cohesion_factor').

La funzione gestisce anche il caso in cui il comportamento toroidale sia abilitato, in questo caso la distanza tra i boids è calcolata attraverso la funzione 'toroidal_distance', definita in 'functions.hpp', e la velocità è indirizzata di conseguenza. In entrambi i casi, la funzione restituisce un vettore che rappresenta la direzione e la forza con cui il boid corrente dovrebbe muoversi per avvicinarsi al centro di massa percepito dello stormo.

separazione. La funzione 'separation' implementa la regola di separazione per i boid, che ha l'obiettivo di mantenere una distanza minima tra ciascun boid e i suoi vicini, evitando collisioni. Questa regola contribuisce a distribuire i boid nello spazio, impedendo che si aggregino eccessivamente. La funzione scorre tutti i boid nello stormo. Se un boid è troppo vicino al boid in esame ('b') e si trova a una distanza inferiore o uguale alla distanza minima ('min_distance'), viene calcolato un vettore correttivo. Questo vettore è diretto lontano dal boid vicino ed è proporzionale alla distanza tra i due boid, normalizzata per assicurare che il vettore correttivo sia più grande quanto più i boid sono vicini.

Se il comportamento toroidale è attivo, la funzione calcola la distanza toroidale tra i boid, tenendo conto della possibilità che i boid si "teletrasportino" attraverso i bordi dello schermo. Infine, la funzione restituisce il vettore correttivo 'c', scalato da un fattore di separazione ('separation_factor') per modulare l'intensità della spinta.

La funzione 'bounce' gestisce il comportamento dei boids in caso di collisione, gestendo gli scontri come urti parzialmente anelastici.

allineamento. La funzione 'alignment' implementa la regola di allineamento per i boid, che permette a ciascun boid di tendere a muoversi nella stessa direzione dei suoi vicini. Questa regola contribuisce a creare un comportamento coordinato all'interno dello stormo, con i boid che cercano di allineare la loro velocità con quella dei boid circostanti. La funzione esamina tutti i boid nello stormo. Se un boid è vicino a 'b' e si trova entro una certa distanza di visuale ('sight_distance'), la velocità del boid vicino viene aggiunta al vettore 'pv', e il contatore 'count' viene incrementato.

Se il comportamento toroidale è attivo, la funzione calcola la distanza toroidale tra i boid. Dopo aver considerato tutti i boid vicini, se 'count' è maggiore di zero, significa che ci sono boid vicini la cui velocità può influenzare quella di 'b'. In questo caso, 'pv' viene diviso per 'count' per ottenere la velocità media percepita dei vicini. La funzione restituisce poi un vettore che rappresenta la distanza media relativa rispetto a 'b' scalato da un fattore di allineamento ('alignment_factor').

paura. la funzione 'fear', attiva nel momento in cui viene impostata la presenza di un predatore, gestisce il tentativo di evasione dei volatili dello stormo dal predatore. Se questi si

trova all'interno della distanza di visuale 'sight_distance' la funzione restituisce una velocità di evasione 'evade_vector' che ha come direzione quella della distanza fra boid e predatore e con modulo pari alla velocità massima del boid a cui viene sottratto un fattore pari al rapporto fra la distanza predatore-boid e la distanza di visuale del boid, il tutto scalato per un fattore di paura ('fear_factor').

La classe contiene una variabile cooldown che rappresenta il tempo passato dall'ultima uccisione.

comportamento al bordo. La funzione 'border', definita nel file 'statistics.cpp', gestisce il comportamento del boid al bordo. Nel caso in cui lo spazio toroidale sia attivo, se un boid supera i limiti orizzontali (assi x e y), viene spostato all'interno dell'area di simulazione, ricomparendo sul lato opposto. In altre parole, se un boid supera il bordo destro, riappare sul bordo sinistro, e viceversa. Per quanto riguarda l'asse verticale (z), la funzione prevede un comportamento di rimbalzo. Se il boid supera i limiti verticali, la sua velocità viene invertita e attenuata per simulare l'impatto, impedendogli di uscire oltre l'altezza massima o sotto il livello minimo. Infine, la nuova posizione e velocità del boid vengono aggiornate.

Nel caso in cui non sia attivo lo spazio toroidale, se un boid raggiunge i bordi dello schermo in una qualsiasi direzione (x, y o z) e la sua velocità lo porterebbe a uscire dai limiti, la funzione inverte la componente corrispondente della velocità, riducendola del 10% per simulare una perdita di energia al momento dell'impatto. Inoltre, la posizione del boid viene corretta per assicurarsi che rimanga all'interno dei confini, evitando che si sposti oltre i limiti. La funzione aggiorna quindi la velocità e la posizione del boid per riflettere queste modifiche.

altezza preferenziale. La funzione 'maintainHeight', anch'essa definita in 'statistics.hpp', fa sì che ogni boid dello stormo applichi delle correzioni al proprio movimento per mantenersi a un'altezza preferenziale ($\frac{2}{3}$ dello schermo come impostazione di default). In particolare, se il boid si trova a un'altitudine diversa da quella predefinita e la sua velocità verticale lo porterebbe ad allontanarsi da tale quota, viene applicata una correzione proporzionale alla sua distanza dall'altitudine preferenziale e scalata mediante un fattore 'height_factor'.

3.2 Le funzioni ausiliarie

Nel file 'functions.cpp' sono definite alcune funzioni di inizializzazione dei parametri dello stormo e aggiornamento della simulazione

3.2.1 Inizializzazione dei parametri

La funzione 'initializeParameters' riveste un ruolo cruciale nella configurazione iniziale della simulazione dei boid. Questa funzione consente di stabilire i parametri di base che determinano il comportamento dello stormo e le condizioni ambientali in cui esso si muove. In particolare, l'utente ha la possibilità di abilitare o disabilitare la presenza del vento e di uno spazio toroidale, il che influenza significativamente il comportamento globale del sistema. La funzione supporta sia un'inizializzazione automatica dei parametri, con valori predefiniti, sia un'inizializzazione manuale, dove l'utente può specificare parametri all'interno di range predefiniti come la dimensione dello stormo, la velocità massima dei boid, la distanza minima tra essi, e fattori di separazione, coesione, e allineamento. Inoltre, è possibile impostare parametri relativi al predatore, come la velocità e il raggio d'attacco.

La funzione offre anche la possibilità di generare i parametri in modo casuale richiamando 'casualParameters' che genera per essi valori casuali all'interno dei range di validità. Questa flessibilità consente di esplorare una vasta gamma di scenari simulativi, rendendo la funzione fondamentale per esperimenti e analisi del comportamento emergente dei boid.

3.2.2 Aggiornamento della simulazione

La funzione 'updateSimulation' rappresenta il cuore dinamico della simulazione. Essa si occupa di aggiornare le posizioni e le velocità sia del predatore sia dei boid in base alle regole comportamentali implementate e agli eventuali input ambientali. Inoltre, questa funzione esegue una stampa periodica delle statistiche della simulazione, come la distanza media tra i boid e la loro velocità media, oltre al loro numero. Questo aggiornamento costante delle statistiche è fondamentale per monitorare l'andamento della simulazione nel tempo e per permettere un'analisi quantitativa del comportamento emergente dello stormo.

3.2.3 Gestione dell'Aspetto Grafico dei Boid

La rappresentazione visiva della simulazione è gestita principalmente attraverso le funzioni 'drawWindows', 'drawBoids', e 'handleEvents'. Queste funzioni, che sfruttano la libreria SFML, permettono di creare, gestire e aggiornare le finestre grafiche in cui viene visualizzata la simulazione.

Creazione e Gestione delle Finestre: La funzione 'drawWindows' è responsabile della creazione delle finestre di visualizzazione. Utilizzando la risoluzione del desktop dell'utente ('getDesktopMode'), questa funzione calcola le dimensioni appropriate per le finestre, tenendo conto delle dimensioni dello schermo e di eventuali elementi dell'interfaccia, come la barra delle applicazioni. Due finestre vengono create per visualizzare il volo dei boid da diverse prospettive: la proiezione XY e la proiezione XZ. Le finestre sono posizionate in modo tale da sfruttare al meglio lo spazio disponibile sullo schermo, facilitando così l'osservazione simultanea delle diverse proiezioni.

Disegno dei Boid e del Predatore: La funzione 'drawBoids' si occupa del disegno effettivo dei boid e del predatore all'interno delle finestre create. I boid vengono rappresentati come cerchi bianchi, con un raggio corrispondente all'apertura alare, mentre il predatore, più grande, è disegnato come un cerchio rosso per distinguerlo chiaramente dal resto dello stormo. La posizione dei boid e del predatore all'interno delle finestre è calcolata in base alle dimensioni dello schermo e alle coordinate relative del piano selezionato.

Gestione degli Eventi: La funzione 'handleEvents' gestisce l'interazione tra l'utente e le finestre della simulazione. Utilizzando un loop di eventi, questa funzione verifica se l'utente ha chiuso una delle finestre e, in tal caso, interrompe il programma. Questo permette una gestione ordinata della simulazione e garantisce che il programma si chiuda in modo controllato quando l'utente decide di interrompere la simulazione.

3.3 Il file main (boids.cpp) e l'interazione con l'utente

Nel file principale 'boids.cpp', l'interazione con l'utente è gestita attraverso una serie di richieste di input definite nei file 'functions.hpp' e 'swarm.hpp'. Questi file contengono messaggi predefiniti che guidano l'utente nella configurazione della simulazione, chiedendo se preferisce includere effetti come il vento o lo spazio toroidale, o se desidera impostare manualmente i parametri. Una volta raccolte le preferenze dell'utente, boids.cpp utilizza le classi e i metodi della libreria SFML per creare e gestire tre finestre che visualizzano le proiezioni ortogonali dello spazio di simulazione. Queste finestre vengono aggiornate in tempo reale tramite il metodo 'updateSimulation', che incrementa un contatore intero 't', permettendo così il progresso continuo della simulazione.

4. Testing del Programma

Per verificare il corretto funzionamento del programma sviluppato, sono stati eseguiti diversi test utilizzando la libreria Doctest. I test sono stati progettati per coprire tutte le principali funzionalità delle classi e delle funzioni implementate, assicurandosi che il comportamento sia quello previsto.

Inoltre il funzionamento del simulatore è stato testato variando i parametri della simulazione e acquisendo le statistiche di output.

4.1 Test della Classe 'Vec3'

1. Costruttori: È stato verificato che il costruttore di default inizializza i valori a zero, mentre il costruttore parametrizzato assegna correttamente i valori passati.

- Input: '*Vec3* v1; *Vec3* v2(1, 2, 3);'
- Output atteso: '*v1*(0, 0, 0); *v2*(1, 2, 3);'

2. Operatori di uguaglianza e disuguaglianza: I test hanno confermato che gli operatori '==' e '!=' funzionano correttamente.

- Input: '*Vec3* v1(1, 2, 3); *Vec3* v2(1, 2, 3); *Vec3* v3(4, 5, 6);'
- Output atteso: '*v1* == *v2*; *v1* != *v3*;'

3. Operazioni di somma e sottrazione: Le operazioni di somma, sottrazione e assegnazione sono state testate per verificare che i risultati siano corretti.

- Input: '*Vec3* v1(1, 2, 3); *Vec3* v2(4, 5, 6); *Vec3* v3 = v1 + v2;'
- Output atteso: '*v3*(5, 7, 9);'

4. Moltiplicazione e divisione scalare: Sono state testate la moltiplicazione e la divisione di un vettore per uno scalare, verificando la correttezza dei risultati.

- Input: '*Vec3* v1(1, 2, 3); *Vec3* v2 = v1 * 2;'
- Output atteso: '*v2*(2, 4, 6);'

5. Norma e normalizzazione: La funzione 'norm()' è stata testata per calcolare correttamente la lunghezza del vettore, mentre 'normalize()' è stata verificata per produrre un vettore unitario.

- Input: '*Vec3* v1(3, 4, 0);'
- Output atteso: '*norm* = 5; *normalize* = *Vec3*(0.6, 0.8, 0.0);'

6. Prodotti scalare e vettoriale: Sono stati eseguiti test per verificare i risultati dei prodotti scalare ('dot') e vettoriale ('cross').

- Input: '*Vec3* v1(1, 0, 0); *Vec3* v2(0, 1, 0);'
- Output atteso: '*dot* = 0; *cross* = *Vec3*(0, 0, 1);'

4.2 Test della Classe 'Boid'

1. Costruttore e accessori: Il costruttore di default e parametrizzato della classe 'Boid' è stato testato per verificare che inizializzi correttamente le posizioni e le velocità dei boid.

- Input: '*Boid* b1; *Boid* b2(*Vec3*(1, 2, 3), *Vec3*(4, 5, 6));'
- Output atteso: '*b1.get_position()* == *Vec3*(0, 0, 0); *b2.get_velocity()* == *Vec3*(4, 5, 6);'

2. Operazioni sui boid: Sono state verificate le funzioni di aggiornamento della posizione e della velocità dei boid, compreso il rispetto della velocità massima.

- Input: '*b.updateBoidVelocity(delta_v, max_speed);'*

- Output atteso: `'b.get_velocity().norm() <= max_speed;'`

3. Uguaglianza e disuguaglianza: Gli operatori `'=='` e `'!='` sono stati testati per verificare la corretta comparazione tra boid.

- Input: `'Boid b1(pos1, vel1); Boid b2(pos2, vel2);'`

- Output atteso: `'b1 == b2;'`

4.3 Test delle Funzioni Globali

2. Media e deviazione standard: Le funzioni `'mean()'` e `'dev_std()'` sono state testate con vettori di valori per verificare che i risultati siano corretti.

- Input: `'std::vector<double> values = {1.0, 2.0, 3.0, 4.0, 5.0};'`

- Output atteso: `'mean = 3.0; dev_std = std::sqrt(2.0);'`

4.4 Test della Classe 'Swarm'

1. Inizializzazione dello stormo: Sono stati eseguiti test sull'inizializzazione della classe `'Swarm'` per verificare che i parametri predefiniti e personalizzati siano correttamente gestiti.

- Input: `'Swarm swarm;'`

- Output atteso: `'swarm.get_size() == 100;'`

2. Interazione tra boid nello stormo: È stato verificato che l'aggiornamento dello stormo con la presenza di un predatore modifichi correttamente il numero e la disposizione dei boid.

- Input: `'swarm.update_swarm(predator);'`

- Output atteso: `'swarm.get_size() <= 10;'`

3. Comportamento ai confini: Sono stati testati i comportamenti di confine per garantire che i boid rispettino i limiti dello spazio o si avvolgano correttamente in modalità toroide.

- Input: `'b.set_position(Vec3(-1, 50, 50)); swarm.border(b);'`

- Output atteso: `'b.get_position().x == 0;'`

4.5 Test della Classe 'Predator'

1. Costruttore e comportamento: È stato verificato che il predatore sia inizializzato correttamente e che le sue funzioni di aggiornamento influenzino lo stormo come previsto.

- Input: `'Predator predator;'`

- Output atteso: `'predator.get_position() == Vec3(0, 0, 0);'`

2. Aggiornamento del predatore: I test hanno confermato che il predatore aggiorna la sua velocità e posizione per inseguire i boid nello stormo.

- Input: `'predator.update_predator(swarm);'`

- Output atteso: `'predator.get_velocity().normalize() == Vec3(1, 0, 0);'`

4.6 Simulazione al variare dei parametri

In questa sezione presentiamo alcuni risultati ottenuti variando i parametri della simulazione

4.6.1 Test 1: Simulazione senza vento e con i bordi

Input:

- Spazio toroidale: Disattivato
- Numero di boid: 120
- Velocità massima: 75
- Parametri di separazione, allineamento, coesione e paura: 82%, 86%, 67%, 98%
- Vento: Assente

Output:

Stormo che pian piano diventa coeso velocemente e poi si muove lentamente nello spazio tridimensionale senza disgregarsi. Nel momento in cui il predatore attacca lo stormo scappa.

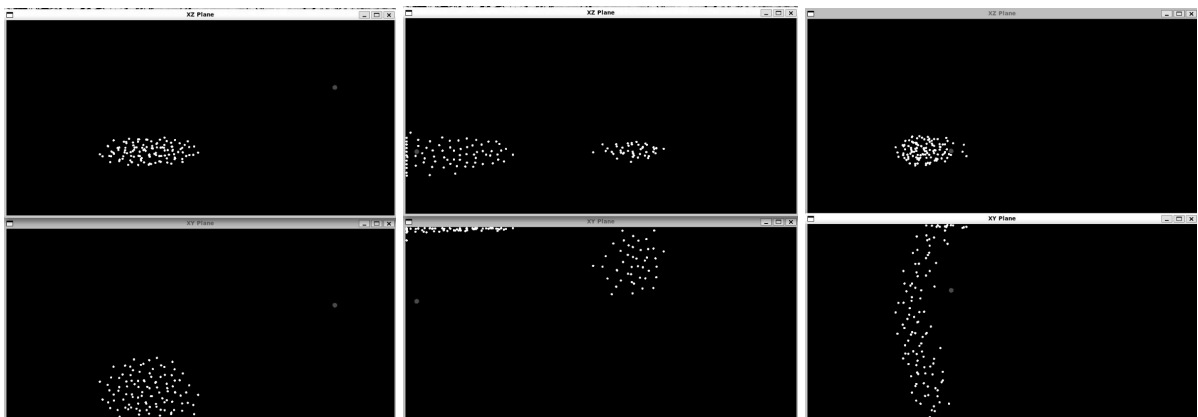
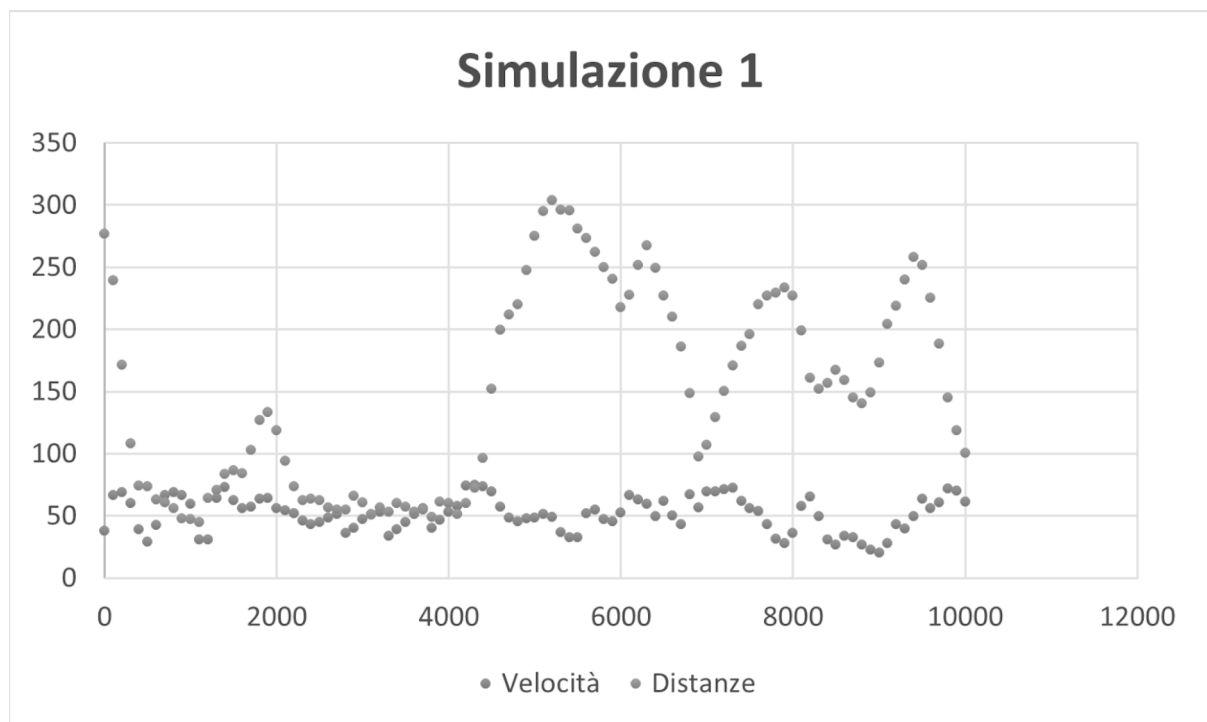


Figura 1: Screenshot della simulazione ai tempi: 4000, 6700, 8700.



4.6.2 Test 2: Simulazione con vento e spazio toroidale

Input:

- Spazio toroidale: Attivato
- Numero di boid: 70
- Velocità massima: 115
- Parametri di separazione, allineamento e coesione: 37%, 52%, 6%, 5%
- Vento: Presente

Output:

- Stormo che si muove in un ampio spazio con traiettoria influenzata dal vento, alcuni boid sono catturati ma lo stormo reagisce lentamente alla presenza del predatore.

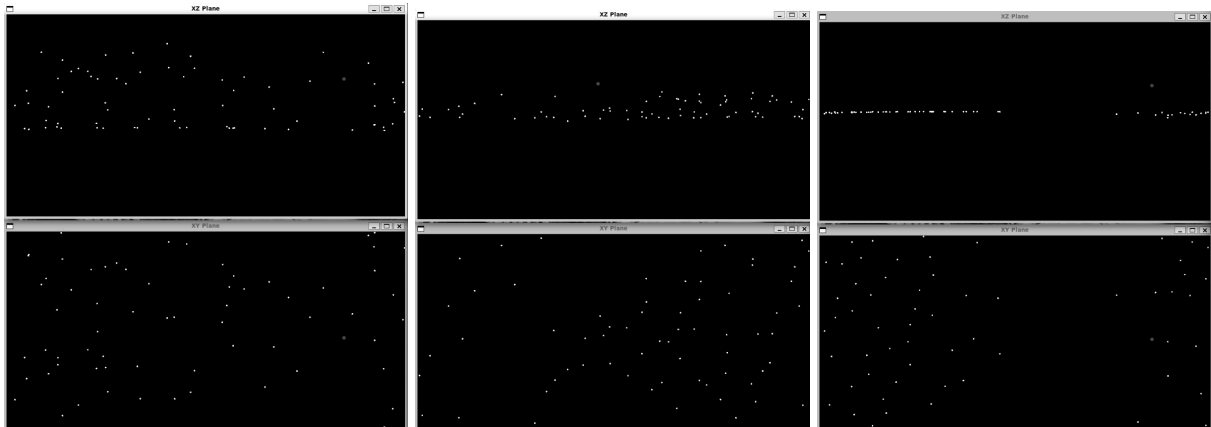
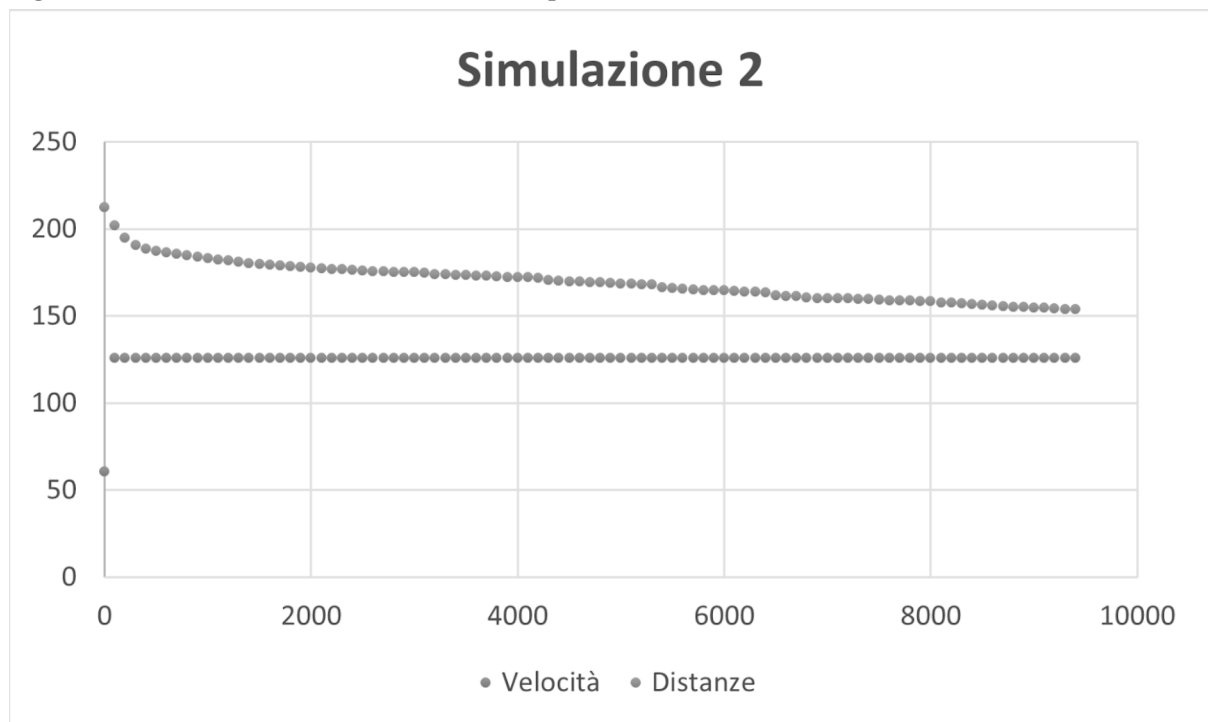


Figura 2: Screenshot della simulazione ai tempi: 700, 4100, 10400.



4.6.3 Test 3: Simulazione con vento e con bordi

Input:

- Spazio toroidale: Disattivato
- Numero di boid: 150
- Velocità massima: 164
- Parametri di separazione, allineamento, coesione e paura: 73%, 50%, 2%, 100%
- Vento: Presente

Output:

- Boid che arrivano ad una coesione lentamente, influenzati dal vento tendono verso un bordo ma non vi rimangono attaccati dal momento che presentano una forte reazione ogni qualvolta il predatore attacca lo stormo. Alcuni boid sono catturati e rimossi dallo stormo.

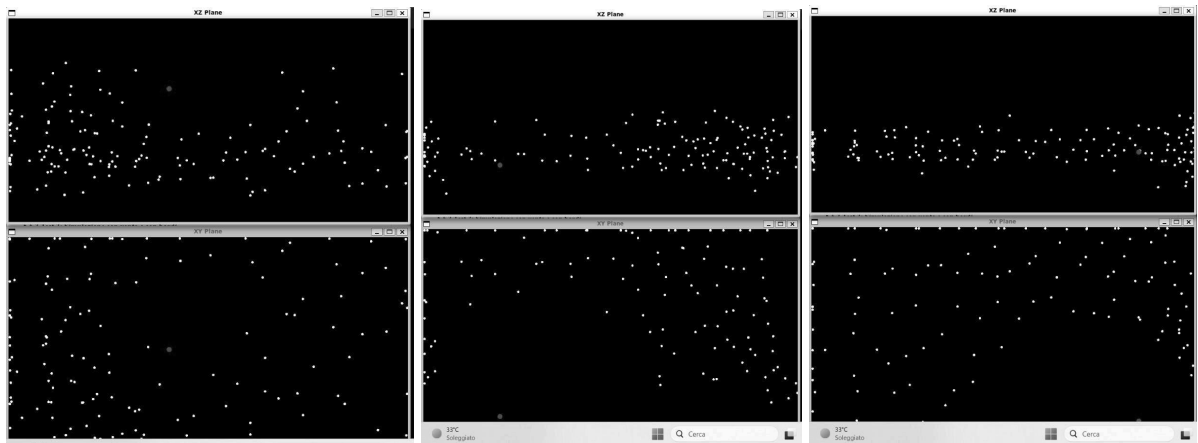
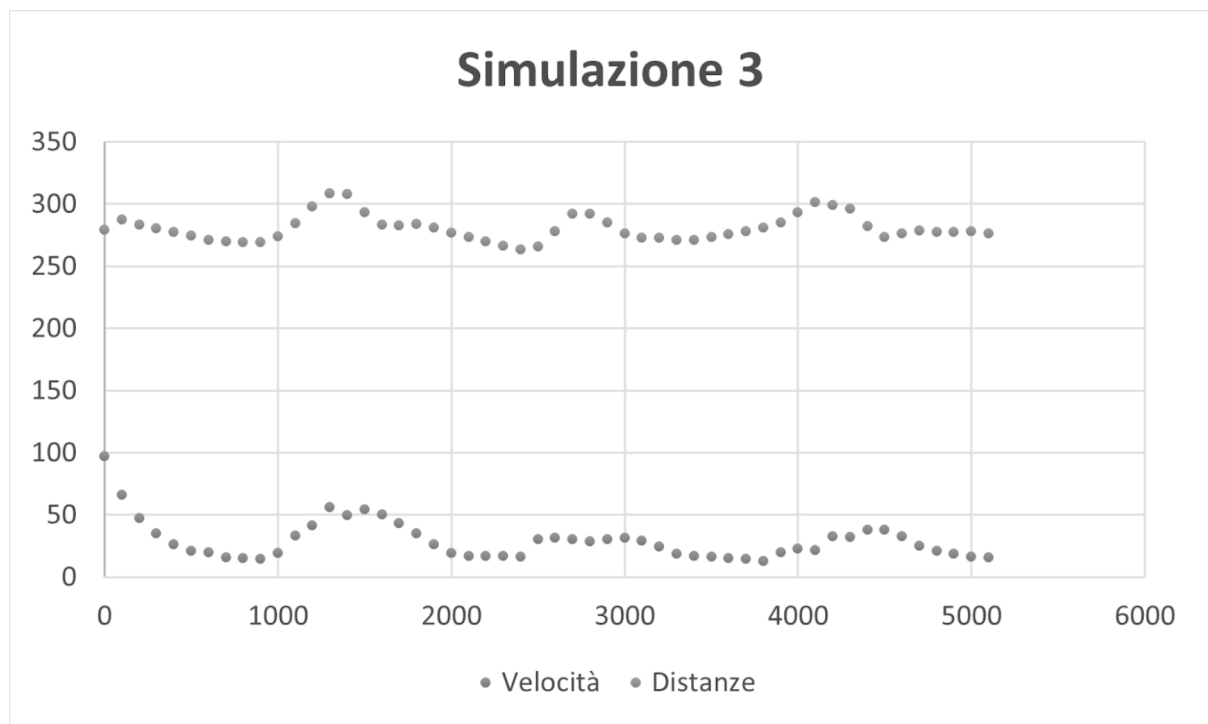


Figura 3: Screenshot della simulazione ai tempi 900, 2600, 4100



4.6.4 Test 4: Simulazione senza vento e con spazio toroidale

Input:

- Spazio toroidale: Attivato
- Numero di boid: 50
- Velocità massima: 179
- Parametri di separazione, allineamento, coesione e paura: 76%, 20%, 66%, 44%
- Vento: Assente

Output:

Boids molto statici, con centro di massa pressoché fisso posizionato “dietro” allo schermo nello spazio toroidale, stormo coeso ma con distanze medie alte e picchi di velocità al momento degli attacchi.

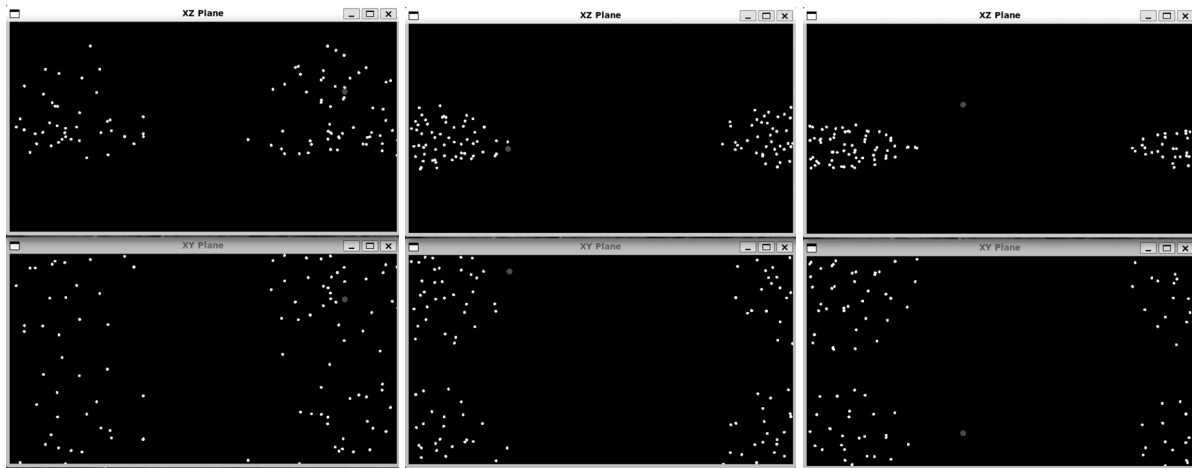
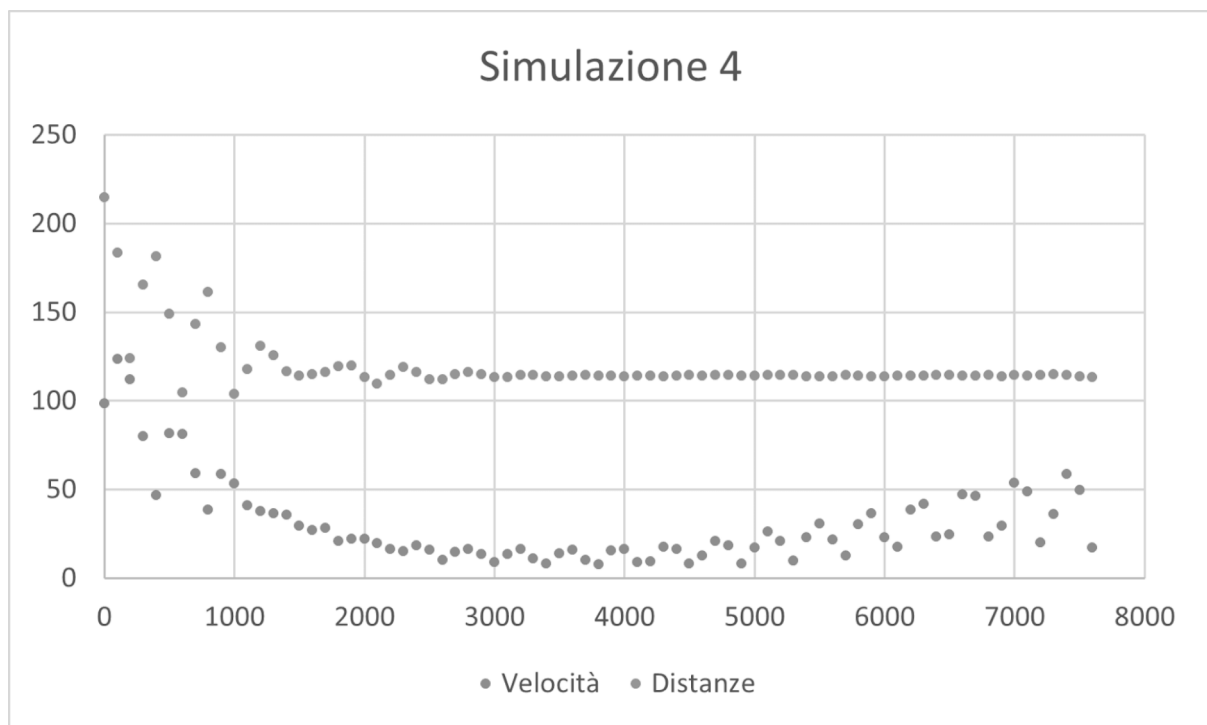


Figura 4: Screenshot della simulazione ai tempi 300, 2100, 5300



Conclusioni

La simulazione si svolge fluidamente con ogni combinazione di variabili provata. L'ampio uso di iteratori non appesantisce significativamente il programma fin quando lo stormo non supera i 150 boid. Lo stormo si comporta come previsto aggregandosi dopo poche iterazioni della funzione *'updateSimulation'*, sia in presenza di vento che non, e in entrambe le possibili configurazioni dello spazio, toroidale e non, inoltre è chiara l'influenza del predatore sul resto dello stormo.

Bibliografia

[1] C. W. Reynolds. Flocks, Herds, and Schools: A Distributed Behavioral Model. Computer Graphics: SIGGRAPH '87 Conference Proceedings, 21(4):25–34, 1987.

[2] <https://vergenet.net/~conrad/boids/pseudocode.html>

Appendice

[3] Repository github contenente il progetto:
<https://github.com/Ciccio-Fonsy/Boids>